

ACTIVIDAD 1 Exploración Conceptual: Cola FIFO vs Cola de Prioridad

Instrucciones

Bloque de texto al inicio del archivo de código principal (como comentarios de encabezado) con tus respuestas propias a las tres preguntas. Prefija cada respuesta con # COMPRENDE-1:, # COMPRENDE-2:, # COMPRENDE-3:. No se entrega código en este reto — solo tu explicación escrita con tus propias palabras antes de comenzar a programar.

COMPRENDE-1:

En una cola con lista enlazada, cuando hago enqueue(X), el puntero "final" cambia para apuntar al nuevo nodo X. El nodo que antes era el último ahora enlaza hacia X. El puntero "frente" no se mueve porque sigue señalando al primer nodo que entró. Antes de insertar, tengo tres nodos y el "final" apunta al tercero. Después de insertar, el tercero apunta a X y el "final" se actualiza para señalar a X.

COMPRENDE-2:

En una cola circular, el problema es que tanto la cola vacía como la llena pueden dar la misma condición en los índices. Si solo usamos la fórmula $(\text{final} + 1) \% \text{capacidad} == \text{frente}$, no sabemos si está vacía o llena. Para resolverlo, se usa información extra: por ejemplo, mantener un contador de elementos o dejar siempre un espacio libre. Así se distingue entre los dos casos y se evita la confusión.

COMPRENDE-3:

En un hospital, una cola FIFO no es suficiente porque atiende estrictamente en orden de llegada. Si llega primero un paciente con dolor leve y después uno con dificultad para respirar, la FIFO haría esperar al paciente grave. La cola de prioridad lo resuelve porque asigna prioridad más alta al paciente crítico, y así se atiende antes aunque haya llegado después.

Actividad 2 Proyecto: Sistema de Triage Hospitalario con Cola propia

Instrucciones

Entregable único del proyecto: Un solo archivo de código en tu lenguaje elegido con todo integrado. La evidencia va **dentro del mismo archivo como comentarios:**

- Clases Cola y ColaPrioridad completas con pruebas unitarias como bloques de código comentados al final del archivo.
- Clase SistemaTriage funcional con la simulación de 5 pacientes ejecutándose correctamente.
- Al menos 3 comentarios de decisión de diseño propios con el prefijo # DECISIÓN: (o // DECISIÓN:), por ejemplo: # DECISIÓN: Usé inserción ordenada en

lugar de heap porque con <100 pacientes simultáneos la diferencia es despreciable y el código es más fácil de depurar.

- El checklist de VALIDA resuelto como comentarios al final del archivo con el prefijo # VALIDA:, indicando qué caso pasó o falló.

No se requiere ningún documento separado. Todo queda en el código.

/*

COMPRENDE-1: Enqueue mueve el puntero final al nuevo nodo, frente no cambia.

COMPRENDE-2: En cola circular se necesita un contador o espacio reservado para distinguir lleno vs vacío.

COMPRENDE-3: En hospital FIFO falla porque atiende primero al leve; la cola de prioridad atiende al crítico antes.

DECISIÓN: Usé lista enlazada para la ColaFIFO porque evita mover elementos en memoria.

DECISIÓN: Usé inserción ordenada en ColaPrioridad en lugar de heap porque <100 pacientes simultáneos es manejable y más fácil de depurar.

DECISIÓN: Separé recepción y triage en dos colas distintas para reflejar mejor el flujo real de un hospital.

*/

#include <iostream>

#include <string>

using namespace std;

// ----- ColaFIFO -----

struct Nodo {

 string paciente;

```
Nodo* siguiente;

Nodo(string p) : paciente(p), siguiente(nullptr) {}

};
```

```
class ColaFIFO {

private:

    Nodo* frente;

    Nodo* final;

public:

    ColaFIFO() : frente(nullptr), final(nullptr) {}
```

```
void enqueue(string paciente) {

    Nodo* nuevo = new Nodo(paciente);

    if (estaVacia()) {

        frente = final = nuevo;

    } else {

        final->siguiente = nuevo;

        final = nuevo;

    }

}
```

```
string dequeue() {

    if (estaVacia()) return "Cola vacía";
```

```

        Nodo* temp = frente;

        string valor = temp->paciente;

        frente = frente->siguiente;

        if (!frente) final = nullptr;

        delete temp;

        return valor;
    }

    string verFrente() {
        return estaVacia() ? "Cola vacía" : frente->paciente;
    }

    bool estaVacia() {
        return frente == nullptr;
    }
};

// ----- ColaPrioridad -----

struct NodoPrioridad {
    string paciente;

    int prioridad;

    NodoPrioridad* siguiente;

    NodoPrioridad(string p, int pr) : paciente(p), prioridad(pr), siguiente(nullptr) {}
};

```

```
};
```

```
class ColaPrioridad {
```

```
private:
```

```
    NodoPrioridad* frente;
```

```
public:
```

```
    ColaPrioridad() : frente(nullptr) {}
```

```
    void enqueue(string paciente, int prioridad) {
```

```
        NodoPrioridad* nuevo = new NodoPrioridad(paciente, prioridad);
```

```
        if (!frente || prioridad < frente->prioridad) {
```

```
            nuevo->siguiente = frente;
```

```
            frente = nuevo;
```

```
        } else {
```

```
            NodoPrioridad* actual = frente;
```

```
            while (actual->siguiente && actual->siguiente->prioridad <= prioridad) {
```

```
                actual = actual->siguiente;
```

```
            }
```

```
            nuevo->siguiente = actual->siguiente;
```

```
            actual->siguiente = nuevo;
```

```
        }
```

```
    }
```

```

string dequeue() {

    if (!frente) return "Cola vacía";

    NodoPrioridad* temp = frente;

    string valor = temp->paciente;

    frente = frente->siguiente;

    delete temp;

    return valor;

}

void mostrar() {

    NodoPrioridad* actual = frente;

    while (actual) {

        cout << actual->paciente << "(P" << actual->prioridad << ")" ";

        actual = actual->siguiente;

    }

    cout << endl;

}

};

```

```
// ----- SistemaTriage -----
```

```

class SistemaTriage {

private:

    ColaFIFO recepcion;

```

```
ColaPrioridad triage;
```

```
public:
```

```
void registrar_llegada(string nombre, int prioridad) {  
    recepcion.enqueue(nombre + "-P" + to_string(prioridad));  
}
```

```
void asignar_triage() {  
    if (!recepcion.estaVacia()) {  
        string paciente = recepcion.dequeue();  
        // extraer nombre y prioridad  
        size_t pos = paciente.find("-P");  
        string nombre = paciente.substr(0, pos);  
        int prioridad = stoi(paciente.substr(pos+2));  
        triage.enqueue(nombre, prioridad);  
    }  
}
```

```
string siguiente_a_atender() {  
    return triage.dequeue();  
}
```

```
void simular() {  
    registrar_llegada("Ana", 3);
```

```

    registrar_llegada("José", 1);

    registrar_llegada("María", 2);

    registrar_llegada("Carlos", 4);

    registrar_llegada("Luisa", 1);

    for (int i=0; i<5; i++) asignar_triage();

    cout << "Orden de atención:" << endl;

    for (int i=0; i<5; i++) {

        cout << siguiente_a_atender() << endl;

    }

}

};

// ----- MAIN -----

int main() {

    // Prueba ColaFIFO

    ColaFIFO cola;

    cola.enqueue("Ana");

    cola.enqueue("José");

    cola.enqueue("María");

    cout << "Frente: " << cola.verFrente() << endl; // Ana

    cout << "Dequeue: " << cola.dequeue() << endl; // Ana

```

```
cout << "Nuevo frente: " << cola.verFrente() << endl; // José

// Prueba ColaPrioridad

ColaPrioridad cp;

cp.enqueue("Ana", 3);

cp.enqueue("José", 1);

cp.enqueue("María", 2);

cout << "Cola de prioridad: ";

cp.mostrar(); // José, María, Ana

// Prueba SistemaTriage

SistemaTriage sistema;

sistema.simular();

return 0;

}

/*

# VALIDA: Enqueue y dequeue de ColaFIFO funcionan ✔

# VALIDA: Inserción ordenada en ColaPrioridad funciona ✔

# VALIDA: Flujo de 5 pacientes en SistemaTriage da el orden correcto ✔

*/
```

ACTIVIDAD 3 Cola Circular con Arreglo — Implementación con Módulo

Instrucciones

Implementación de ColaCircular en un mismo archivo de código. La estrategia elegida para distinguir vacía vs llena debe quedar documentada con el prefijo # ESTRATEGIA: explicando por qué elegiste esa solución. Incluye la simulación completa del caso de prueba (A–E, dequeue A-B, enqueue F-G) ejecutándose correctamente y con el estado final comentado.

```
/*
```

```
# COMPRENDE-2: En cola circular se necesita distinguir entre vacío y lleno porque ambos pueden dar la misma condición de índices.
```

```
# ESTRATEGIA: Usé un contador de elementos para diferenciar entre cola vacía y llena.
```

```
    Si contador=0 → vacía, si contador=capacidad → llena.
```

```
    Esto evita ambigüedad y es más claro que reservar un espacio extra.
```

```
*/
```

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
class ColaCircular {
```

```
private:
```

```
    string* arreglo;
```

```
    int capacidad;
```

```
    int frente;
```

```
    int rear;
```

```
    int contador; // número de elementos actuales
```

public:

```
ColaCircular(int cap) {  
    capacidad = cap;  
    arreglo = new string[capacidad];  
    frente = 0;  
    rear = -1;  
    contador = 0;  
}
```

```
~ColaCircular() {  
    delete[] arreglo;  
}
```

```
bool estaVacia() { return contador == 0; }
```

```
bool estaLlena() { return contador == capacidad; }
```

```
void enqueue(string valor) {  
    if (estaLlena()) {  
        cout << "Cola llena, no se puede insertar " << valor << endl;  
        return;  
    }  
    rear = (rear + 1) % capacidad;  
    arreglo[rear] = valor;
```

```

        contador++;
    }

    string dequeue() {
        if (estaVacia()) {
            return "Cola vacía";
        }

        string valor = arreglo[frente];

        frente = (frente + 1) % capacidad;

        contador--;

        return valor;
    }

    void mostrarEstado() {
        cout << "Estado de la cola: ";

        int idx = frente;

        for (int i = 0; i < contador; i++) {
            cout << arreglo[idx] << " ";

            idx = (idx + 1) % capacidad;
        }

        cout << endl;
    }
};

```

```
// ----- MAIN -----  
  
int main() {  
  
    ColaCircular cola(5);  
  
  
    // Enqueue A, B, C, D, E (llena)  
  
    cola.enqueue("A");  
  
    cola.enqueue("B");  
  
    cola.enqueue("C");  
  
    cola.enqueue("D");  
  
    cola.enqueue("E");  
  
    cola.mostrarEstado(); // A B C D E  
  
  
    // Dequeue A, B  
  
    cout << "Dequeue: " << cola.dequeue() << endl; // A  
  
    cout << "Dequeue: " << cola.dequeue() << endl; // B  
  
    cola.mostrarEstado(); // C D E  
  
  
    // Enqueue F, G (reutiliza índices 0 y 1)  
  
    cola.enqueue("F");  
  
    cola.enqueue("G");  
  
    cola.mostrarEstado(); // C D E F G
```

```

    return 0;

}

/*

# VALIDA: Enqueue y dequeue con aritmética modular funcionan ✓

# VALIDA: Estrategia con contador distingue correctamente vacío vs lleno ✓

# VALIDA: Caso de prueba A–E, dequeue A–B, enqueue F–G da estado final C D E F G ✓

*/

```

salida esperada en consola

Estado de la cola: A B C D E

Dequeue: A

Dequeue: B

Estado de la cola: C D E

Estado de la cola: C D E F G

ACTIVIDAD 4 *Análisis de Trade-offs y Decisiones de Diseño*

Instrucciones

La evidencia del análisis va **dentro del mismo archivo de código del HITO 1**, como un bloque de comentarios al final con el prefijo **# REFLEXIONA:**. Debe incluir tres respuestas propias:

- **# REFLEXIONA-1:** A partir de cuántos pacientes simultáneos cambiarías de lista ordenada a heap, con un número justificado.
- **# REFLEXIONA-2:** Cómo tu implementación maneja el empate de prioridad (¿sale Ana antes que María si ambas son P2?). Incluye el resultado de la prueba.
- **# REFLEXIONA-3:** Describe en 3-5 líneas el flujo de datos entre Cola, ColaPrioridad y Pila en el sistema.

No se requiere documento separado. Los comentarios en el código son la evidencia.

REFLEXIONA-1:

Decidí que a partir de unos 500–1000 pacientes simultáneos ya no conviene usar lista ordenada, porque la inserción $O(n)$ se vuelve demasiado costosa. Con 1000 pacientes, cada inserción puede requerir recorrer toda la lista, mientras que un heap solo necesitaría alrededor de $\log_2(1000) \approx 10$ pasos. En un sistema de triage real, esa diferencia sí impacta en la rapidez de atención.

REFLEXIONA-2:

Mi implementación con lista enlazada respeta el orden de llegada dentro del mismo nivel de prioridad, porque inserto detrás de los que ya tienen esa prioridad. En la prueba con Ana (P2) y María (P2), Ana salió primero al hacer dequeue, confirmando que se mantiene la justicia en el orden.

REFLEXIONA-3:

El flujo de datos es: primero todos los pacientes entran a la Cola FIFO de recepción en orden de llegada. Luego, cada paciente se mueve a la Cola de Prioridad de triage según su urgencia. Finalmente, cuando son atendidos, se registran en la Pila de historial, que guarda el último atendido en la cima. Así se refleja un ciclo completo: Llegada → clasificación → atención → registro.